

Programming Assignment description and resources 2026

Programming assignment intended outcome

The objective of this assignment is to enable the student to continue self-learning socket programming by practicing it on a real project.

In particular, we want the student to learn how to deploy concurrent servers; how to deploy multi-threaded socket programs to enable clients to asynchronously send and receive data; and how to implement the rdt3.0 on UDP. For this, the student would need to learn individually (with the occasional help from the instructors) how to use the POSIX thread (pthread) library.

Programming assignment object

Build a server-based multiparty chatroom

Description

Your objective is to implement a client-server application to create a multiparty chatroom. In the beginning, the server side starts and waits for connection requests to arrive from clients. When a client starts, it requests immediately a connection to the server, upon which it is required to supply a nickname for the user in the chatroom. The nickname is used by the server for registration or for login. The server maintains a list (or array) of **user nicknames** and their associated **client socket file descriptors** as well as the handles to their offline message boxes.

Our chatroom can handle several commands as well as two modes of communication as described below. Once clients are connected, the server acts as a proxy between different clients.

A client can send one of several requests to the server, which the server interprets, responds to appropriately:

- REGISTER : is sent in response to the server prompting the user to supply a nickname for registration or login.
- DIR: can be sent by the client to the server to retrieve the nicknames, IP addresses, port numbers and the online status of all other users in the system (excluding itself), and display the nicknames and online status of all users on the screen.
- EXIT: can be sent by the client to the server to leave the chatroom

A client can decide to send a broadcast message to all by simply typing the message (like in zoom chatroom), or it can decide to send a message to another client privately. For this latter, the client must type:

#<Nickname>: <MSG>,

where <Nickname> is the nickname of the destination client and <MSG> is the message we want to send privately. If the destination client is online, the message will be sent to it directly using UDP with the implemented RDT3.0. If the client is offline, or UDP-RDT transfer fails because of a timeout the message will be stored on the server and relayed to the destination client by the server upon its next login. The time space diagrams for these exchanges are illustrated on the powerpoint slides of the last lab.

Requirements:

- Programming must be in C
- You must use `poll()` system call to handle multiple clients concurrently, instead of `select()` or `epoll()`. Otherwise, you will get no points.
- Each client must supply a unique nickname for registration and login. If the nickname supplied already exists, we regard it as a login case. Otherwise, regard it as a registration case.
- Clients must be able to send private messages to other clients and send public broadcast messages.
- The online private message need to be sent with UDP Protocol. And for the reliability, you need to implement the RDT 3.0 on UDP.
- The offline private message must be store in the server, and need to be forwarded when the destination client login. Use TCP to implement it.
- The broadcast message must first be sent to the server, which then forwards the message to all other online clients. Both communications use TCP.
- Clients should be able to send and receive messages simultaneously. That is we must use the POSIX thread library *pthread* to create a thread for sending and one for receiving messages.
- Both the client and the server must handle exceptions and errors accordingly.

Skeletons for the client and the server are provided for you. You need to implement the missing parts as indicated inside the files by `/*****/`.

Grading scheme

- Registration – 5pt
- Direct messages to online users with UDP – 10pt
- Implement the RDT3.0 on UDP - 25pt
- Message box for offline user - 10pt
- Exit – 5pt

- Broadcast messages – 5pts
- Multi-user – 10pts
- Simultaneously receiving and sending (multi-threads) – 10pt
- Report – 20pt

Your program will be tested and graded using the `grader_linux.sh` script provided for your usage. If you are not using Linux, you can install the latest Ubuntu on VirtualBox and run the Linux script to simplify your life. Windows users may install WSL and run the test within the WSL environment. Note that you cannot use the script on the lab machines as it requires superuser privilege when testing RDT to create packet losses.

Additionally, our grading script redirects the output to a log file for verification. Therefore, your message output format must match the demo version and slides published on Canvas to avoid confusion. You can also refer to the script file for confirmation of the output format. Furthermore, please use `fflush(stdout)` after outputting the message to ensure immediate writing to the file.

To facilitate our grading and testing, please do not rename your code files.

Grader Script Mapping:

- **Test 1 (Basic Operations):** Covers Registration (5pt), Exit (5pt), Multi-user (10pt). Total: 20 pts.
- **Test 2 (Broadcast & Async I/O):** Covers Broadcast messages (5pt), Simultaneously receiving and sending (10pt). Total: 15 pts.
- **Test 3 (Reliability & P2P):** Covers Direct messages to online users with UDP (10pt), Implement the RDT3.0 on UDP (25pt). Total: 35 pts.
- **Test 4 (Offline Message Fallback):** Covers Message box for offline user (10pt). Total: 10 pts.
- *(Remaining 20 pts are evaluated from your final report).*

Deductions: points will be deducted for functionalities not implemented and for bugs not handled by each part on its own.

Bonus Points

To earn **5 extra bonus points**, you can implement a Command Line Interface GUI (using the ncurses library) where the screen is divided into three panes. On the left, a column from top to bottom to display the list of online friends. In the bottom, a one-line pane for the user to input messages, and the rest of the screen is used to output messages received from the server and the clients.

Crash introduction to pthread: what you need to know to 'hit the ground running'

pthread library is the POSIX responses to threads (i.e, the standard for threads on all the Unix-like systems)

- to link against it, you compile your program by appending `-lpthread` at the end of your compile command.
- to use it, you must include `pthread.h` by adding `#include <pthread.h>`
- to create a new thread, you declare the thread, you create a function to execute as the thread's code, then you make a system call to `create_thread` to actually create the thread. example:

```
void *print_message_function( void *ptr ); /* define the thread function */

pthread_t thread1, thread2; /* declare two threads */

...
iret1 = pthread_create( &thread1, NULL, print_message_function, (void*) message1);
// create thread 1 to execute the print_message_function,
// with argument message1

iret2 = pthread_create( &thread2, NULL, print_message_function, (void*) message2);
// create thread 2 to execute the print_message_function,
// with argument message2
```

The above code creates two threads `thread1` and `thread2`. they are associated with the function 'print_message_function' and in the first instance the function takes `message1` as an input argument whereas in the second instance it takes `message2`.

Consult the man page for `pthread_create`, by typing `>man pthread_create` on the shell command line

- To terminate a thread we can call `pthread_exit()`
- `pthread` treats the `main()` function of a C program as a thread and will schedule it to run concurrently with other threads that are explicitly created via `pthread_create`. As such if two threads share some data and can modify it concurrently, we have to use some synchronisation mechanisms to ensure the access to the data is serialised otherwise we may run into a race condition. Usually we avoid this by locking the sections of code that modify the shared data (we call such code a critical section). We always have to be careful if our code shares data and has critical sections. To avoid problems we can use a mutual exclusion (mutex) variable to manage synchronisation and avoid a race condition.

To use a mutex for mutual exclusion,

we can declare and initialize a mutex by:

```
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
```

then in the code, wherever we execute a critical section, we must surround the code by

```
pthread_mutex_lock(&mutex1);  
  
<critical section>  
  
pthread_mutex_unlock(&mutex1);
```

this would enable our code in the critical section to execute uninterrupted by only one thread at a time until the section is finished, **however, you must be careful** to ensure that the code in the critical section has a limited time horizon, i.e., if you call a blocking function (that may go to sleep) then you must ensure that ultimately it would wake up and progress, otherwise, you enter into a deadlock.

If you forgot your COMP3511 or did not take that course, you can consult the pthread documentation as well as the many tutorials that exist on the Internet. Or ultimately, you can ask us questions on canvas chatroom.